NAME : Kinjal Srivastava SAP ID: 500123394

BATCH: 1

NAME :Naman Singh SAP ID: 500123185

BATCH: 2

Final Project Report

AWS-Based Task Reminder CLI Tool - Event Triggering Notification using Lambda and SNS in Java

1. Introduction

This project presents the development of a command-line interface (CLI) tool built using Java, designed to help users manage and receive timely reminders for their tasks. The motivation behind this project stems from the increasing need for minimal, real-time productivity solutions that are easy to integrate and deploy across environments. By leveraging Amazon Web Services (AWS) such as Lambda, SNS, EC2, and S3, the tool efficiently delivers email notifications for upcoming task deadlines. The application is designed to be lightweight, scalable, and serves as a foundational piece for future integrations, especially with larger systems like the proposed "Virtual Study Room" app.

2. Project Goals & Objectives

- Build a Java-based command-line application for managing task reminders.
- Enable real-time email notifications using AWS Lambda and Simple Notification Service (SNS).
- Store task metadata as text files in Amazon S3 for backup and audit purposes.
- Demonstrate environment provisioning and deployment using Amazon EC2.
- Maintain modularity and extensibility to support integration with future tools, such as a virtual study platform.

3. AWS Services Used & Configurations

The tool utilizes several AWS services to accomplish its goals.

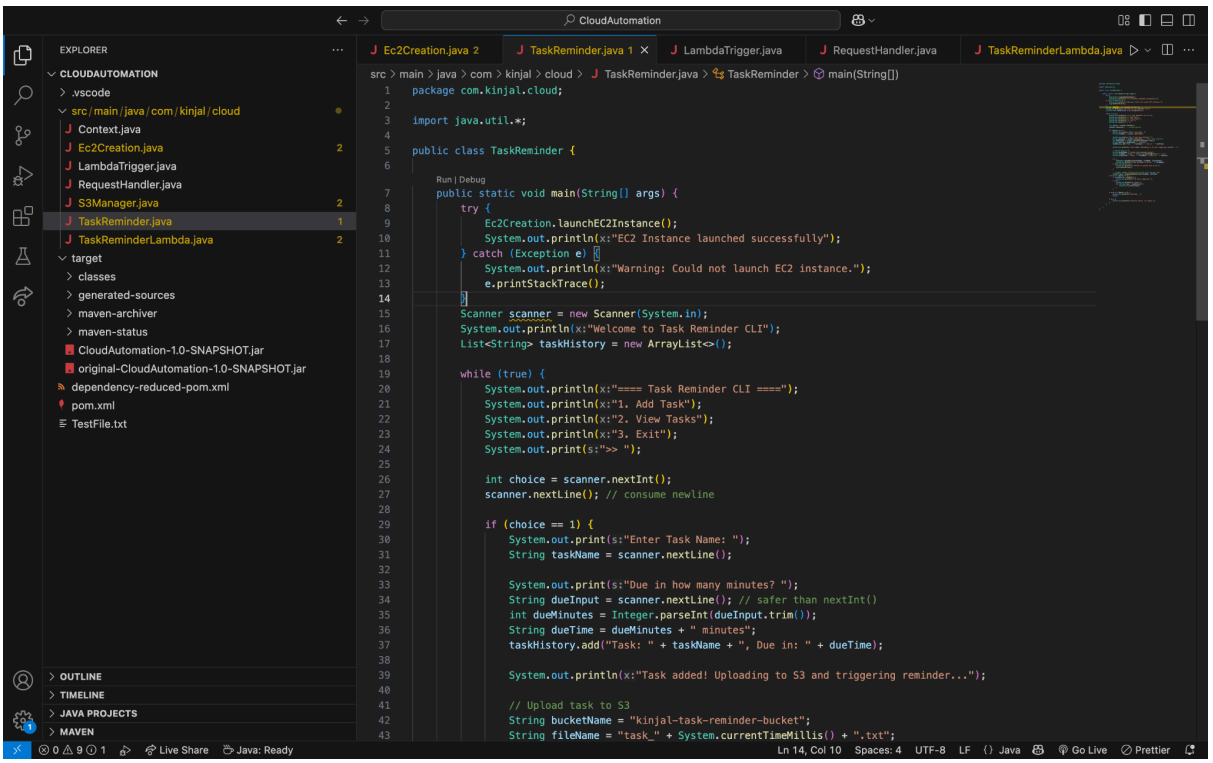
AWS Lambda was configured using the Java 11 runtime, with the handler method set to com.kinjal.cloud.TaskReminderLambda::handleRequest. The Lambda function was packaged and deployed as a ZIP file containing the application JAR.

Amazon SNS (Simple Notification Service) was used to create a topic specifically for sending email alerts, named task-reminder-project. These alerts are triggered whenever a task deadline is approaching.

Amazon EC2 was used for demonstrating the infrastructure provisioning process, such as simulating application deployment in a cloud-based virtual machine during presentations or testing.

Amazon S3 served as the storage solution for saving task-related metadata in the form of plain text files. This allows future auditing and acts as a simple backup solution.

4. Implementation Details & Code Snippets



Main Application Workflow

The core application class serves as the entry point of the CLI-based task reminder tool. When executed, it performs the following key functions:

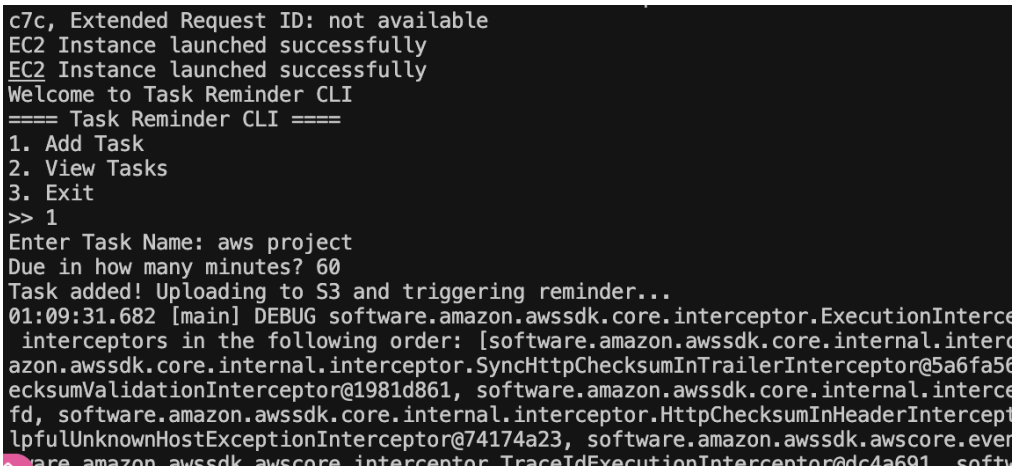
1. **Launches an EC2 Instance**:
   Upon startup, the application initiates an Amazon EC2 instance to simulate deployment in a cloud environment. This demonstrates the application's compatibility with cloud infrastructure and its potential for scalable deployment.

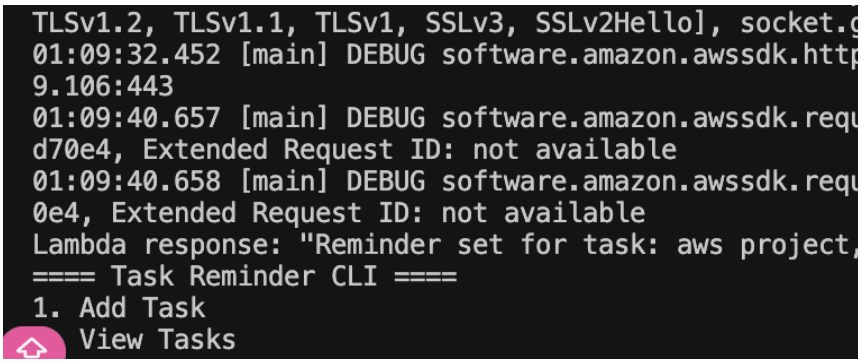
2. **Menu-Driven Interface**:
   The program enters a looped, menu-driven mode to interact with the user via the command line. It provides three options:

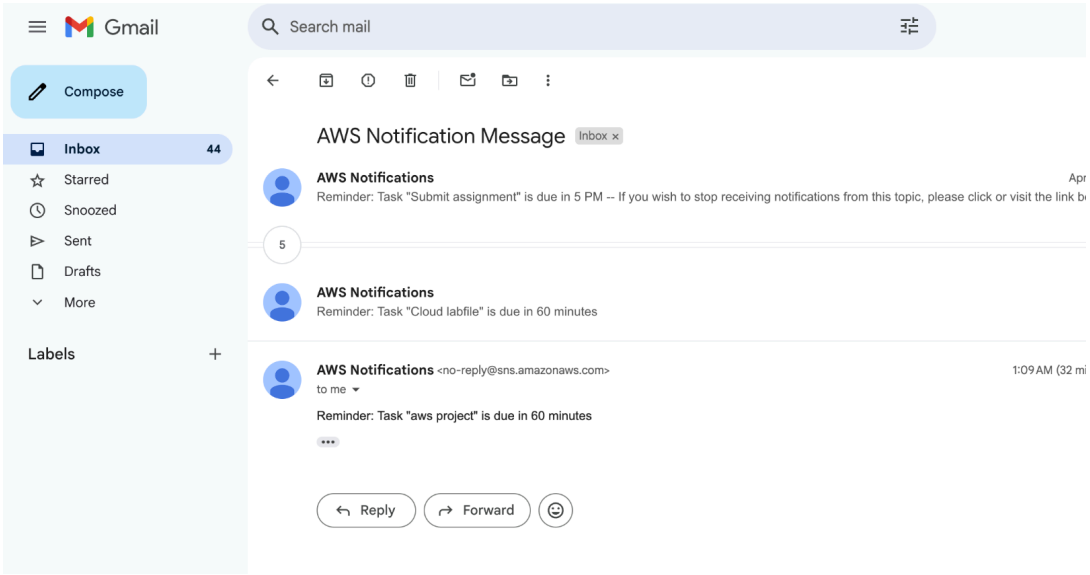
   - **Option 1: Add Task**
     When the user selects this option, the application prompts for task details such as task name and due time. It then:

     - Creates an Amazon S3 bucket (if not already present).
     - Stores the task metadata as a text file in the S3 bucket.
     - Invokes the AWS Lambda function with the task details as input.

- The Lambda function, in turn, publishes a notification through Amazon SNS, sending an email reminder to the user.

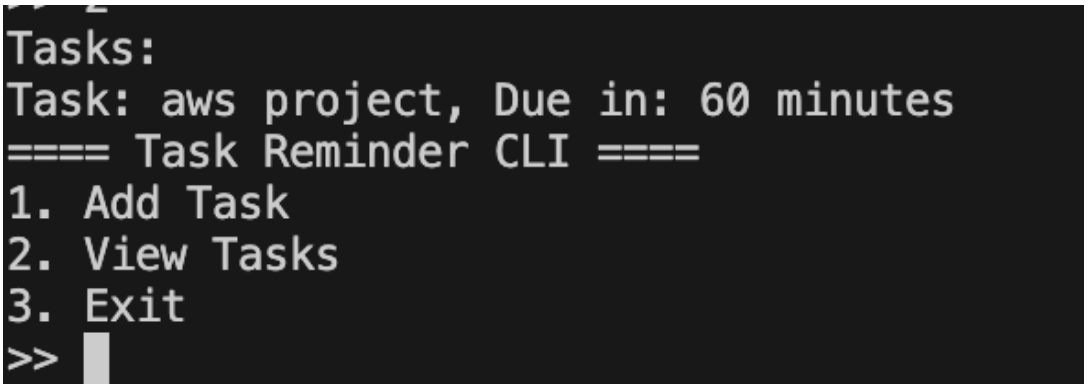




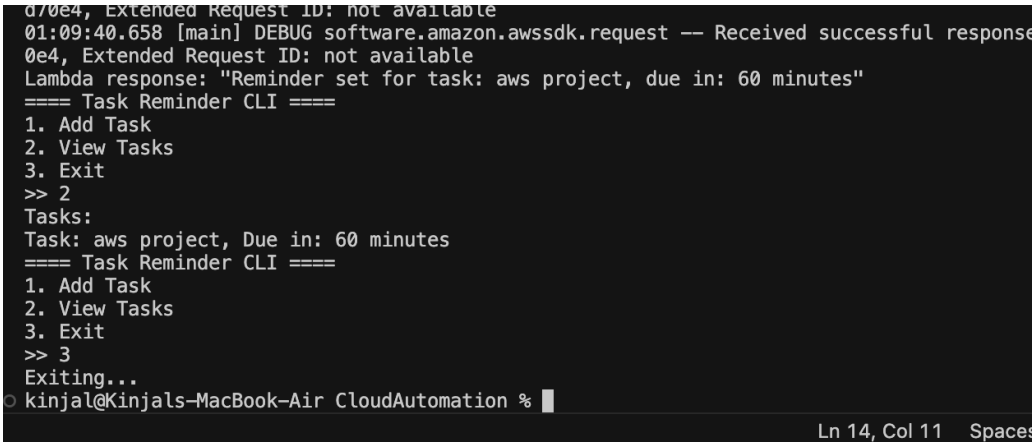


- **Option 2: View Tasks**
  This option displays all tasks added during the current session, which are maintained locally in a list (typically an ArrayList or similar data structure). It

serves as a quick way for users to review their current tasks without accessing S3.



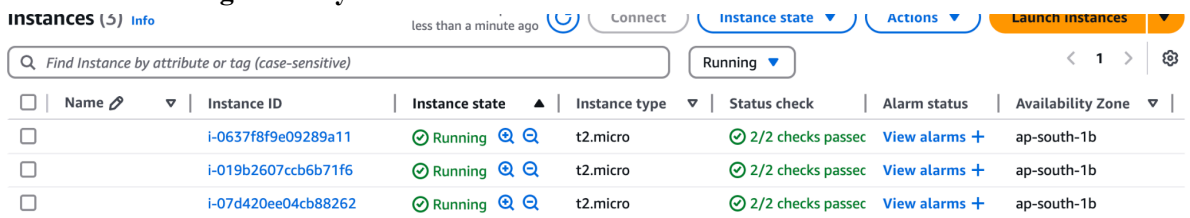
- **Option 3: Exit**
  Selecting this option terminates the menu loop and safely exits the program.



This structured flow not only demonstrates integration with AWS services but also offers a user-friendly way to interact with the application through a simple command-line interface.

Following our Services implementation classes:

EC2 creation :Pragmatically



```java
package com.kinjal.cloud;

import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.*;

import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.*;

public class Ec2Creation {
    public static void launchEC2Instance() {
        Ec2Client ec2 = Ec2Client.create();

        RunInstancesRequest runRequest = RunInstancesRequest.builder()
                .imageId(imageId:"ami-0f1dcc636b69a6438") // Use correct AMI for your region
                .instanceType(InstanceType.T2_MICRO)
                .maxCount(maxCount:1)
                .minCount(minCount:1)
                .build();

        ec2.runInstances(runRequest);
        System.out.println(x:"EC2 Instance launched successfully");
    }
}
```

S3 manager to create and upload task files:

```java
package com.kinjal.cloud;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.*;

import java.io.File;
import java.nio.file.Path;
import java.nio.file.Paths;

public class S3Manager {

    private static final String testBucket = "kinjal-task-reminder-bucket";

    // Create an S3 client using profile credentials
    private static S3Client s3Client = S3Client.builder()
            .credentialsProvider(ProfileCredentialsProvider.create()) // Use ProfileCredentialsProvider
            .region(software.amazon.awssdk.regions.Region.AP_SOUTH_1) // Set your region
            .build();

    // Create a new S3 bucket
    public static void createBucket(String bucketName) {
        try {
            CreateBucketRequest createBucketRequest = CreateBucketRequest.builder()
                    .bucket(bucketName)
                    .build();

            s3Client.createBucket(createBucketRequest);
            System.out.println("Bucket created: " + bucketName);

        } catch (S3Exception e) {
            e.printStackTrace();
        }
    }

    // Upload a file to the specified S3 bucket
    public static void uploadFile(String bucketName, String key, String filePath) {
        try {
            Path path = Paths.get(first:"TestFile.txt");
            PutObjectRequest putObjectRequest = PutObjectRequest.builder()
                    .bucket(bucketName)
                    .key(key)
                    .build();
```

Lambda Class to invoke and trigger Lambda Function and SNS Notification:

```java
import java.util.Map;

public class TaskReminderLambda implements RequestHandler<Map<String, String>, String> {

    private static final String TOPIC_ARN = "arn:aws:sns:ap-south-1:222634404865:task-reminder-project";

    @Override
    public String handleRequest(Map<String, String> input, Context context) {
        String task = input.get(key:"task");
        String dueTime = input.get(key:"dueTime");

        String message = "Reminder: Task \"" + task + "\" is due in " + dueTime;

        try (SnsClient snsClient = SnsClient.create()) {
            PublishRequest request = PublishRequest.builder()
                    .message(message)
                    .topicArn(TOPIC_ARN)
                    .build();

            PublishResponse result = snsClient.publish(request);
            System.out.println("Message sent! ID: " + result.messageId());
        }

        return "Reminder set for task: " + task + ", due in: " + dueTime;
    }
}
```

```java
public class LambdaTrigger {

    private static final String REGION = "ap-south-1"; // Set your AWS region
    private static final String LAMBDA_FUNCTION_NAME = "taskReminderLambda"; // Your Lambda function name

    // Create a Lambda client
    private static LambdaClient lambdaClient = LambdaClient.builder()
            .credentialsProvider(ProfileCredentialsProvider.create()) // Use profile credentials
            .region(software.amazon.awssdk.regions.Region.of(REGION))
            .build();

    // Method to invoke the Lambda function
    public static void invokeLambdaFunction(String task, String dueMinutes) {
        try {
            // Example payload
            String payload = String.format(format:"{\"task\": \"%s\", \"dueTime\": \"%s\"}", task, dueMinutes);

            InvokeRequest invokeRequest = InvokeRequest.builder()
                    .functionName(LAMBDA_FUNCTION_NAME) // Lambda function name
                    .payload(SdkBytes.fromUtf8String(payload)) // Pass the payload
                    .build();

            InvokeResponse invokeResponse = lambdaClient.invoke(invokeRequest);

            String response = invokeResponse.payload().asUtf8String(); // Get the response as a string
            System.out.println("Lambda response: " + response);

        } catch (LambdaException e) {
            e.printStackTrace();
        }
    }
}
```

5. Challenges Faced & Solutions

During the development and deployment of the AWS-based CLI Task Reminder Tool, several technical challenges were encountered:

- **Lambda ClassNotFoundException**
  Initially, when deploying the Lambda function, a `ClassNotFoundException` was thrown. This was caused by incorrect packaging of the Lambda JAR file. The issue was resolved by using Maven's `clean package` command to ensure that all dependencies were properly bundled into the JAR. Additionally, the handler string was double-checked to match the fully qualified class and method name.

- **Issues with JAR and ZIP Files for Lambda Deployment**
  There was confusion and difficulty in preparing the correct deployment artifact for the Lambda function. AWS Lambda requires the deployment package (ZIP) to include the compiled JAR with all dependencies, and the directory structure must be preserved. Initially, the function either failed silently or threw errors during execution. The solution was to use the Maven Shade plugin to create a fat JAR (also known as an uber JAR), which includes all dependencies in a single file. This JAR was then zipped and uploaded as the Lambda deployment package, resolving the issue.

- **Manual Setup Before Maven Migration**
  At the beginning of the project, all AWS SDK JAR files were manually downloaded and referenced directly from the terminal without using any build tool. This led to challenges in dependency management, project structure, and build consistency. Transitioning the project to Maven was initially difficult due to unfamiliarity with `pom.xml` configuration and plugin setup. However, once established, Maven streamlined the entire build process, allowing automatic dependency resolution and easier packaging for AWS Lambda.

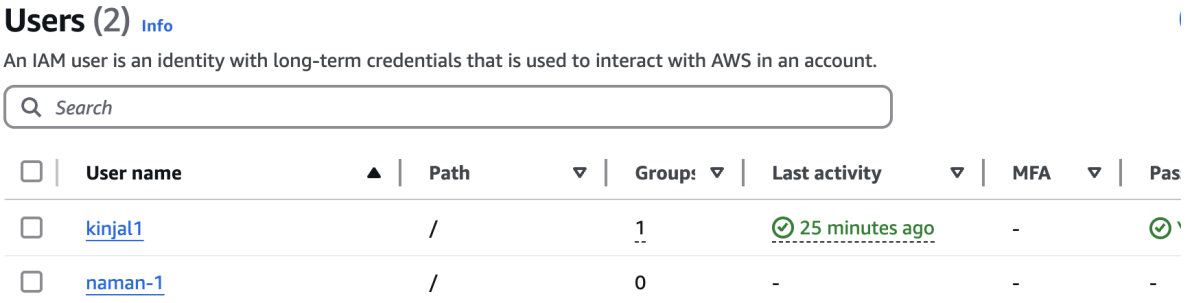
- **SNS Not Sending Dynamic Task Info**
  The email notifications were initially sending default or static messages regardless of the task entered by the user. This was traced back to the LambdaTrigger component, which was not passing dynamic payloads. Updating the trigger logic to send user-inputted task data as part of the Lambda event resolved this issue.

- **Reminder Trigger Outside Task Flow**
  The Lambda invocation was originally placed outside the actual task-adding logic, resulting in reminders being sent even when no task was added. To fix this, the invocation was moved inside the task creation block, ensuring reminders are only triggered when a new task is submitted.

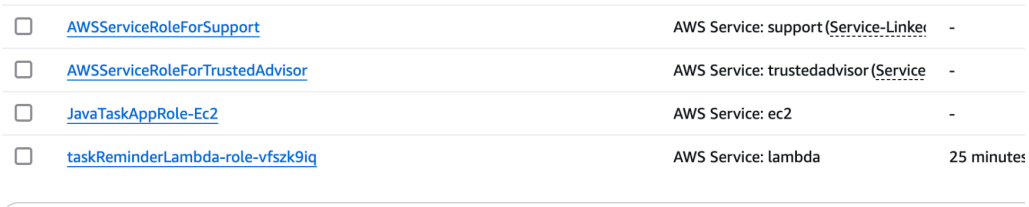
6. Security Considerations

Security was prioritized throughout the development of this tool to ensure safe interaction with AWS services:

- Two IAM users, **Kinjal** and **Naman**, were configured during development for access control and testing purposes. These separate identities allowed secure collaboration while maintaining individual accountability.



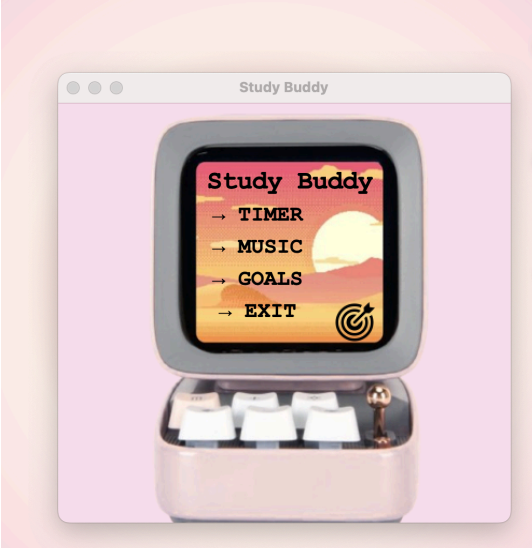
- IAM roles were created and assigned specifically to AWS services used in the project:
    - The **Lambda role** was restricted to only publish messages to the designated SNS topic (`task-reminder-project`), following the principle of least privilege.
    - The **EC2 instance role** was granted limited access, ensuring it could only perform necessary operations such as logging or interacting with S3, without broader permissions.

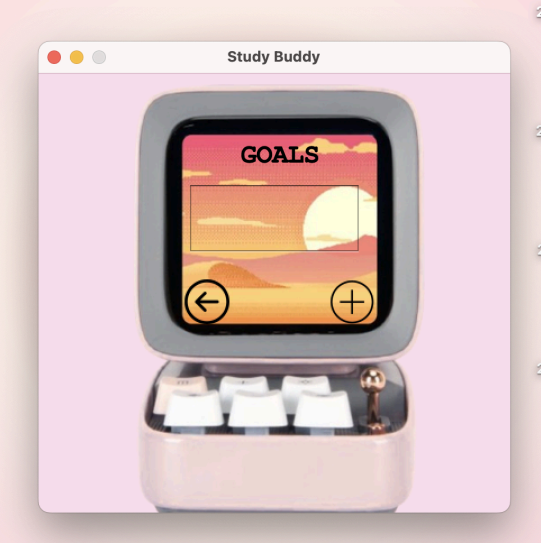


- AWS credentials were securely managed using the AWS credentials profile stored locally, avoiding any hardcoded secrets in the codebase.
- The **SNS Topic ARN** and other sensitive configuration details were externalized as constants rather than being hardcoded, promoting safer and more maintainable code.

7. Future Improvements & Scalability

This CLI tool sets the foundation for several possible enhancements. One of the most promising directions is its integration with the Virtual Study Room application(Java and Java Swing used), where the reminder system can enhance the To-Do list feature with real-time email alerts.





Additionally, future versions may utilize Amazon CloudWatch Events to schedule future reminders more flexibly. Moving from simple S3 storage to DynamoDB would allow better querying, editing, and personalized task management.

Finally, introducing user authentication via AWS Cognito or a similar service would make it possible to associate tasks with individual users, enabling a fully managed personal task reminder system.

Conclusion

The AWS-Based Task Reminder CLI Tool successfully demonstrates the power and flexibility of integrating cloud services into a lightweight Java application. By leveraging AWS Lambda for logic execution, SNS for real-time notifications, S3 for storage, and EC2 for deployment simulation, the project not only meets its original goal of task reminders but also lays a strong foundation for future extensibility.

Despite facing initial hurdles—such as manual setup, deployment packaging issues, and IAM configurations—the project evolved into a secure, scalable, and modular application. The transition to Maven, use of appropriate IAM roles, and dynamic task handling significantly improved its maintainability and usability.

Looking ahead, this tool can be seamlessly integrated into larger applications like the planned Virtual Study Room, with additional features such as user authentication, database support, and web-based UI interfaces. This project not only enhanced technical skills in cloud computing and Java but also offered practical experience in designing real-world, cloud-native applications.